

Mar 02, 2025 11:49

ControlDB_new.py

Page 1/21

```
1  # this controls the database interactions in pyside6
2  import sys
3  import os
4  import platform
5  from loguru import logger
6
7  import numpy as np #for pyinstaller
8  import config_mycal as CM
9  import IngredientTable as INTA
10
11  from Recipe1 import Ui_MainWindow
12
13  from missing_ingredient import Ui_MissingIngredient
14  from missing_ingredient_dialog import Ui_missing_ingredient_dialog
15  from pick_swiss_ingreds import Ui_Dialog
16  import pandas as PD # to pack the recipe information into a pandas dataframe
17  from IngredientTable import IngredientTable
18
19
20  from PySide6.QtCore import (QSize, Qt ,QRect,
21  QCoreApplication,Slot,Signal,QAbstractListModel,QAbstractTableModel,QMetaObject,QModelIndex)
22  from PySide6.QtGui import QAction,QDoubleValidator,QFont
23
24  from PySide6.QtUiTools import QUiLoader
25
26  from PySide6.QtSql import QSqlDatabase , QSql,QSqlTableModel,QSqlQueryModel,QSqlQuery
27
28  from PySide6.QtWidgets import (
29  QApplication,
30  QCheckBox,
31  QComboBox,
32  QDateEdit,
33  QDateTimeEdit,
34  QDial,
35  QDialog,
36  QDoubleSpinBox,
37  QFileDialog,
38  QFontComboBox,
39  QFormLayout,
40  QHBoxLayout,
41  QLabel,
42  QLCDNumber,
43  QLineEdit,
44  QListView,
45  QMainWindow,
46  QWidget,
47  QProgressBar,
48  QPushButton,
49  QRadioButton,
50  QSlider,
51  QSpinBox,
52  QStatusBar,
53  QTimeEdit,
54  QTableView,
55  QMenuBar,
56  QVBoxLayout,
57  QAbstractItemView,
```

Mar 02, 2025 11:49

ControlDB_new.py

Page 2/21

```

58     QGridLayout,
59     QTextEdit
60
61 )
62
63 class NewWindow(QMainWindow):
64     def __init__(self, Title = None):
65         super().__init__()
66         layout = QVBoxLayout()
67
68         self.label = QLabel("Another Window")
69         if(Title == None):
70             self.setWindowTitle("MyCal")
71         else:
72             self.setWindowTitle(Title)
73
74         layout.addWidget(self.label)
75         self.setLayout(layout)
76
77
78 class RecipeModel(QAbstractTableModel):
79
80     def __init__(self, recipes=None, header_labels=None):
81         super().__init__()
82
83         #When subclassing QAbstractListModel, you must provide implementations of the rowCount() and data() functions.
84         # Well behaved models also provide a headerData() implementation.
85         # this means we will have to define the data and rowCount
86         # see also https://doc.qt.io/qt-6/qabstractlistmodel.html
87
88         self.recipes = recipes or [[]] #recipes will be a 2d array or a nested list
89         self.header_labels = header_labels = ["weight", "unit", "ingredient"]
90     def data(self, index, role):
91         if role == Qt.DisplayRole or Qt.ItemDataRole.DisplayRole or role == Qt.ItemDataRole.EditRole:
92             value = self.recipes[index.row()][index.column()]
93             # check for type
94             if isinstance(value, float):
95
96                 return "%.2f" % value
97
98             if isinstance(value, str):
99                 return "%s" % value
100
101
102             return value
103
104
105     def rowCount(self, parent=QModelIndex()):
106         return len(self.recipes)
107
108     def columnCount(self, parent=QModelIndex()):
109         return len(self.recipes[0])
110
111     def headerData(self, section, orientation, role=Qt.DisplayRole):
112         if role == Qt.DisplayRole and orientation == Qt.Horizontal:
113             return self.header_labels[section]
114

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 3/21

```

115         return None
116
117     def insertRows(self, position, rows=1, parent=QModelIndex(), *args, **kwargs):
118         self.beginInsertRows(parent, position, position + rows - 1)
119         for _ in range(rows):
120             self.recipes.insert(position, [""] * self.columnCount())
121         self.endInsertRows()
122         return True
123
124     def setData(self, index, value, role):
125         if role == Qt.ItemDataRole.EditRole:
126             self.recipes[index.row()][index.column()] = value
127
128             #here we stor the index of the field which has been clicked.
129             self.clicked_row = index.row()
130             self.clicked_column = index.column()
131
132         return True
133         return False
134     def flags(self, index):
135         return Qt.ItemFlag.ItemIsSelectable | Qt.ItemFlag.ItemIsEnabled | Qt.ItemFlag.ItemIsEditable
136
137     def StoreValue(self):
138         return self.recipes
139
140
141     class MyRecipeWindow(QMainWindow, Ui_MainWindow):
142         def __init__(self):
143             super().__init__()
144             self.setupUi(self)
145
146     class MyMissIngWindow1(QMainWindow, Ui_MissingIngredient):
147         def __init__(self):
148             super().__init__()
149             self.setupUi(self)
150
151     class MyMissingIngredientDialog(QDialog, Ui_missing_ingredient_dialog):
152         def __init__(self):
153             super().__init__()
154             self.setupUi(self)
155
156         #def accept(self):
157         #     """overwrite the accept function"""
158         #     #print("custom func")
159         #     super().accept()
160
161
162
163
164
165
166
167     class ContrlDB_new(QMainWindow):
168
169         def __init__(self, Title=None, db_name=None, db_user=None, db_system=None, db_pwd = None, config_file = None):
170             super().__init__()
171             self.db_name = db_name

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 4/21

```

172     self.db_user = db_user
173     self.db_system = db_system
174     self.config_file = config_file
175     if db_pwd == None:
176
177         temp = '/Users/klein/git/qt_exercises/config/pw.txt'
178         if os.path.exists(temp):
179             with open(temp, 'r') as file:
180                 password = file.read().rstrip()
181                 self.db_pwd = password
182         else:
183             self.db_pwd = db_pwd
184
185     self.init_variables()
186     self.SetupLogger()
187     #self.CreateIngredientsForm()
188     self.SetupConfig()
189     self.ingredients_suppress_columns = self.CM.ingredients_suppress_columns
190
191     self.recipes_suppress_columns = self.CM.recipes_suppress_columns
192     self.swiss_food_suppress_columns = self.CM.swiss_food_suppress_columns
193
194     self.version = '2.0'
195     logger.info(' ***** \n\n\n')
196
197     logger.info(' This is version %s' % self.version)
198     logger.info(' \n\n\n ***** \n')
199
200     #instantiate the IngredientTable
201     #table_name = '/Users/klein/git/qt_exercises/nutrition_databases/swiss_food.csv'
202     #if we are using the swiss_food table we need to get the IngredientTable class
203     self.swiss_food = False
204     if(self.CM.ingred_table == "swiss_food"):
205         self.swiss_food = True
206         logger.warning("nned work, pass the table")
207         self.InTa = IngredientTable(table_name=self.CM.ingred_table, ingredients = None)
208
209     def SetupConfig(self):
210
211         # get the config filename
212         # here we do a filedialog
213         if(self.config_file == None or not os.path.isfile(self.config_file)):
214             self.config_file , filter = QFileDialog.getOpenFileName(self,
215                 self.tr("Open Config file"), "~", self.tr("*.json"))
216
217
218         logger.info("config file %s" % self.config_file)
219
220
221
222         self.CM = CM.MyConfig(self.config_file)
223         self.log_level = self.CM.log_level
224         self.log_output = self.CM.log_output
225
226
227     def init_variables(self):
228         """ initialize some variable to None """

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 5/21

```

229
230     self.missing_ingredient = None
231     self.new_recipe_name = None
232
233     def ConnectDataBase(self):
234         ''' establish contact to database'''
235
236
237         #instantiate the connection
238         self.mycal_db = QSqlDatabase.addDatabase(self.db_system)
239         self.mycal_db.setHostName("192.168.2.164")
240         self.mycal_db.setDatabaseName(self.db_name)
241         self.mycal_db.setUserName(self.db_user)
242         self.mycal_db.setPassword(self.db_pwd)
243
244
245     #         self.mycal_db.setPassword(self.CM.pwd)
246
247     result = self.mycal_db.open()
248     if(result):
249         logger.info('connection successful')
250         # here we get the connection name
251         # will be needed when we want to close the connection
252         self.connection_name = self.mycal_db.connectionName()
253         logger.info(' database connection name %s' % self.connection_name)
254         #self.ShowTables()
255     else:
256         logger.error(' database problem %s' % self.mycal_db.lastError().text())
257         logger.error(' connection failed, exiting')
258         sys.exit(0)
259
260     def SetupLogger(self):
261
262
263         #logger.remove(0)
264         #now we add color to the terminal output
265         logger.add(sys.stdout,
266                   colorize = True, format="<green>{time}</green> {function} {line} {level} <level>{message}</level>" ,
267                   level = "DEBUG")
268
269
270
271         fmt = "{time}-{name}- {function}-{line}-{level} -{message}"
272         logger.add('info.log', format = fmt , level = 'INFO', rotation="1 day")
273
274
275         # set the colors of the different levels
276         logger.level("INFO", color = '<black>')
277         logger.level("WARNING", color = '<green>')
278         logger.level("ERROR", color = '<red>')
279         logger.level("DEBUG", color = '<blue>')
280
281     return
282
283
284
285     def ShowTables(self):

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 6/21

```

286         '''prints all the tables in the database'''
287         self.table_list = self.mycal_db.tables(type=QSql.Tables)
288         for k in range(0, len(self.table_list)):
289             print(self.table_list[k])
290
291         #Create list box
292         self.mybox = QComboBox()
293         #Insert at index 0 the whole list
294         self.mybox.insertItems(0, self.table_list)
295
296         # Now sort them alphabetically
297         self.sort_mybox()
298
299         # create independent window
300
301         #self.setCentralWidget(self.mybox)
302
303
304         return
305     def sort_mybox(self):
306         items = [self.mybox.itemText(i) for i in range(self.mybox.count())]
307         items.sort()
308         self.mybox.clear()
309         self.mybox.addItems(items)
310
311     def ViewTable(self, table, suppress_columns=[], editmode=True, columnwidth=[], Title = None, showTable = True):
312         self.table_view = QTableView()
313         self.table_view.setSortingEnabled(True)
314         #self.table_view.sortByColumn(2, Qt.AscendingOrder)
315
316         self.model = QSqlTableModel(db = self.mycal_db)
317
318
319
320         self.table_view.setModel(self.model)
321         #prevent editing
322         if(not editmode):
323             self.table_view.setEditTriggers(QAbstractItemView.NoEditTriggers)
324
325         self.model.setTable(table)
326
327         # get rid of columns
328         columns_to_remove = suppress_columns
329         for cn in columns_to_remove:
330             idx = self.model.fieldIndex(cn)
331             self.model.removeColumns(idx, 1)
332
333
334         self.model.select()
335         # change column width
336         for k in range(0, len(columnwidth)):
337             self.table_view.setColumnWidth(k, columnwidth[k])
338
339         self.setMinimumSize(QSize(600, 300))
340
341         self.setCentralWidget(self.table_view)
342

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 7/21

```

343         self.SetPosition(10,10)
344         if (showTable):
345             self.show()
346         #self.setCentralWidget (table)
347
348
349     def SetPosition (self, pos_x, pos_y):
350         self.move (pos_x, pos_y)
351
352     def CreateIngredientsForm1 (self):
353         """uses the form layout"""
354
355         #always use self so window wll be persistent
356         self.Inform = NewWindow (Title = "Add New Ingredient")
357         self.Inform.setStyleSheet ("background-color: yellow;")
358
359         # setup geometry
360         mysize=[400,300]
361         myposit = [800,100]
362
363         self.SizeWindow (self.Inform, myposit, mysize)
364
365         # Create widgets
366         #fill with ingredients if called from missing ingredients
367         if( self.missing_ingredient != None or self.missing_ingredient != '' ):
368             self.name_label = QLineEdit (self.missing_ingredient)
369             # find position in table where missing ingredient
370
371         else:
372             self.name_label = QLineEdit ()
373             self.name_label.setStyleSheet ("background-color: rgb(3, 252, 227)")
374             # force valid entr
375             double_validator = QDoubleValidator (0, 500.0, 2)
376
377             self.calory_label = QLineEdit ()
378             self.calory_label.setStyleSheet ("background-color: white")
379             self.calory_label.setValidator (double_validator)
380
381             self.carb_label = QLineEdit ()
382             self.carb_label.setStyleSheet ("background-color: white")
383             self.carb_label.setValidator (double_validator)
384             self.fat_label = QLineEdit ()
385             self.fat_label.setStyleSheet ("background-color: white")
386             self.fat_label.setValidator (double_validator)
387             self.prot_label = QLineEdit ()
388             self.prot_label.setStyleSheet ("background-color: white")
389             self.prot_label.setValidator (double_validator)
390
391             SaveButton = QPushButton ("Save")
392             SaveButton.setStyleSheet ("background-color: white")
393             CancelButton = QPushButton ("Cancel")
394             CancelButton.setStyleSheet ("background-color: red")
395
396
397         # Create layout
398         form_layout = QFormLayout ()
399         form_layout.addRow ("Name:", self.name_label)

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 8/21

```

400     form_layout.addRow("Calories/100g", self.calory_label)
401     form_layout.addRow("Fat", self.fat_label)
402     form_layout.addRow("Carbohydrates", self.carb_label)
403     form_layout.addRow("Protein", self.prot_label)
404     form_layout.addRow(SaveButton)
405     form_layout.addRow(CancelButton)
406
407     mylayout = QVBoxLayout()
408     mylayout.addLayout(form_layout)
409     self.setLayout(mylayout)
410
411     # Connect submit button
412     SaveButton.clicked.connect(self.SaveIngredients)
413     CancelButton.clicked.connect(lambda : self.Cancel(self.Inform))
414
415     widget = QWidget()
416     widget.setLayout(mylayout)
417
418
419
420
421
422     self.Inform.setCentralWidget(widget)
423
424     #for label in MyLabel:
425     #    widget = QWidget()
426     #    widget.setLayout(layout)
427     #self.setCentralWidget(widget)
428
429
430     # create the form
431
432     self.Inform.show()
433
434
435     def InsertRecord(self, table = None):
436         """ this inserts a record into the chosen table
437         I will use the query function to do this.
438         The record contains the info to be added
439         record=[name,energy,fat,carbohydrate,protein]
440
441         This is the same for ingredients and receipes """
442
443         #Create table view
444         if(table == "ingredients"):
445
446             self.ViewTable(table = table, suppress_columns=self.ingredients_suppress_columns, showTable= False)
447         elif(table == "recipes"):
448             self.ViewTable(table = table, suppress_columns=self.recipes_suppress_columns)
449         elif(table == "swiss_food"):
450             self.ViewTable(table = table, suppress_columns=self.swiss_food_suppress_columns)
451         else:
452             logger.error("no table given")
453             sys.exit(0)
454
455
456     self.model = QSqlQueryModel()

```


Mar 02, 2025 11:49

ControlDB_new.py

Page 9/21

```

457     self.table_view.setModel(self.model)
458     #next we check we don't have an entry yet
459
460     #Form the sql INSERT statement
461     #     self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
462     #INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);
463     #INSERT INTO table (name,energy,carbohydrate,fat,protein) VALUES (self.record[0],
464     #                                                                    self.record[1],
465     #                                                                    self.record[2],
466     #                                                                    self.record[3],
467     #                                                                    self.record[4]);
468     temp = ''+self.record[0]+'''
469     sql = 'INSERT INTO '+table+' (name,energy,carbohydrate,fat,protein) VALUES ('+temp+','+str(self.record[1])+','+str(self.record[2])+','+str(
self.record[3])+','+str(self.record[4])+');'
470     #print(sql)
471
472     #query = QSqlQuery(sql,db=self.mycal_db)
473
474     #self.model.setQuery(query)
475     self.do_sql(sql)
476     logger.info(' inserted record %s into table %s' % (self.record[0],table))
477
478
479
480     return
481
482
483
484 def MyMissingIngredients(self):
485     """ form if ingredient is missing"""
486     if(not self.swiss_food):
487
488         self.MMI1 = MyMissingIngredientDialog()
489         self.MMI1.IngredLine.setText(self.missing_ingredient)
490         self.MMI1.EditButton.clicked.connect(self.show_ingred_form)
491         #self.MMI1.CloseButton.clicked.connect(self.Cancel(self.MMI1))
492
493
494         self.MMI1.move(100,50)
495         t = self.MMI1.exec()
496
497         #if t == QDialog.Accepted:
498         #     self.show_ingred_form()
499         #     #self.MMI1.close()
500
501
502         self.MMI1.show()
503     else:
504         self.InTa.find_pattern(self.missing_ingredient[0:5])
505         self.InTa.get_ingredient()
506         #self.InTa.get_ingred_selection()
507         # here we put the misssing ingredient back into the form
508         self.RecipeModel.recipes[self.stor_row][self.stor_column] = self.InTa.ingredient_name
509         self.InTa_name = self.InTa.ingredient_name
510         self.InTa_energy = self.InTa.energy
511         self.InTa_carbohydrate = self.InTa.carbohydrate
512         self.InTa_protein = self.InTa.protein

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 10/21

```

513         self.InTa_fiber = self.InTa.fiber
514         self.InTa_sugar = self.InTa.sugar
515         self.InTa_salt = self.InTa.salt
516         self.InTa_fat = self.InTa.fat
517
518
519     return
520
521
522
523 def show_ingred_form(self):
524     # destroy dialog give action back
525     try:
526         self.MMI1.destroy()
527     except:
528         logger.debug("nothing to destroy")
529
530 #         window.CreateIngredientsForm1()
531         self.CreateIngredientsForm1()
532
533
534
535 def MyRecipes_new(self):
536     """using the QTdesigner"""
537     self.MRD = MyRecipeWindow()
538     self.MRD.show()
539
540     # Load the model
541     #self.header_labels = header_labels = ["weight", "unit", "ingredient"]
542
543     self.RecipeModel = RecipeModel()
544     self.MRD.RecipeTableView.setModel(self.RecipeModel)
545
546     # now working self.RecipeModel.setEditStrategy(QSqlTableModel.setEditStrategy.OnFieldChange)
547
548     #label the columns
549     header = self.MRD.RecipeTableView.horizontalHeader()
550
551
552     #poulate the combo box
553     self.MRD.RecipeListCombo.setObjectName(u"RecipeListCombo")
554     self.MRD.RecipeListCombo.addItem(self.GetRecipeList())
555
556
557     #here we define the actions
558     self.MRD.RecipeListCombo.currentTextChanged.connect(self.getSelectedRecipe)
559
560     self.MRD.RecipeTableView.clicked.connect(self.on_item_click)
561
562     # save button
563     self.MRD.SaveRecipe.clicked.connect(self.save_recipe)
564
565     #cancel button
566     self.MRD.CancelButton.clicked.connect(lambda : self.Cancel(self.MRD))
567
568     #close button
569     self.MRD.CloseButton.clicked.connect(lambda : self.Cancel(self.MRD))

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 11/21

```

570
571     #add row button
572     self.MRD.AddRowButton.clicked.connect(self.add_row)
573
574     #new recipe button
575     #self.MRD.NewRecipeButton.clicked.connect(self.create_new_recipe)
576
577     #if line has change
578     self.MRD.NewRecipeName.returnPressed.connect(self.create_new_recipe)
579     #self.MRD.NewRecipeName.textChanged.connect(self.create_new_recipe)
580
581     # here we fill first label
582     self.MRD.version_line.setText(self.version)
583
584     def create_new_recipe(self):
585         """creates new recipe"""
586         #first get name
587         self.new_recipe_name = self.MRD.NewRecipeName.text()
588         logger.info("new recipe name %s" % self.new_recipe_name)
589         #let's create a default starter table
590         #first create the data list for the model
591         self.RecipeModel.recipes = []
592
593         self.recipe_ingredients=[['0','g',' '],['0','g',' '],['0','g',' '],['0','g',' '],['0','g',' ']]
594         self.DisplayRecipeList()
595         self.RecipeModel.layoutChanged.emit()
596
597
598     return
599
600     def create_pandas_frame(self, name=None):
601         """This routine creates the pandas frames which then get passed to CalculateCalories class"""
602         #collect the information
603         logger.info(" name %s and record %s for recipes" % (name, self.panda_ingredients))
604
605         # now we need to get the different quantities for each ingredient
606         # at the same time we need to check if the ingredient is in the list available
607         #if not we have to deal with this
608         number_column = 4
609         energy,protein,carbohydrate,fat = range(number_column) # for referencing the query values
610
611         logger.debug('recipes ingredient: %s' % self.RecipeModel.recipes)
612
613         #example sql statement:
614         # SELECT energy,protein,carbohydrate,fat FROM ingredients where name = 'Butter';
615
616
617         energy_v,protein_v,carbohydrate_v,fat_v = ([ for i in range(number_column))
618         print(self.RecipeModel.recipes)
619
620         # loop over all ingredients
621         Total_calories_per_gm = 0. # cacluated calories/g
622         Total_protein_per_gm = 0.
623         Total_fat_per_gm = 0.
624         Total_carbs_per_gram = 0.
625         temp_weight = 0.
626         Total_weight = 0.

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 12/21

```

627     print(self.panda_ingredients)
628     #remove empty rows
629     temp_buf=[]
630     for k in range(len(self.panda_ingredients)):
631         if self.panda_ingredients[k][2] != '':
632             temp_buf.append(self.panda_ingredients[k])
633
634     self.panda_ingredients= temp_buf
635
636     for k in range(len(self.panda_ingredients)):
637         sql ='SELECT energy,protein,carbohydrate,fat FROM '+self.CM.ingred_table+' where name ILIKE\' ' + self.panda_ingredients[k][2] +'\';'
638         #sql ='SELECT energy,protein,carbohydrate,fat FROM ingredients where name ILIKE \' ' + self.panda_ingredients[k][2] +'\';'
;
639         logger.debug('the query string %s' % sql)
640
641         query = self.do_sql(sql)
642
643         logger.debug('value of query.first %s' % query.first())
644         while query.next():
645
646             temp_value = ((query.value(0)))
647
648     #model = QSqlQueryModel()
649
650     if( not query.first()):
651         logger.info('this ingredient %s is missing'%self.panda_ingredients[k][2])
652         self.missing_ingredient = self.panda_ingredients[k][2]
653         # here we stor the index of the missing ingredient:
654         self.stor_row = self.RecipeModel.clicked_row = k
655         self.stor_column = self.RecipeModel.clicked_column = 2
656
657         self.MyMissingIngredients()
658     else:
659         #fill variables
660         energy_v.append(float(query.value(energy)))
661         protein_v.append(float(query.value(protein)))
662         carbohydrate_v.append(float(query.value(carbohydrate)))
663         fat_v.append(float(query.value(fat)))
664         temp_weight = float(self.RecipeModel.recipes[k][0])/100. # all is normalized to per 100 g
665
666         Total_calories_per_gm = Total_calories_per_gm+temp_weight*energy_v[k]
667         Total_protein_per_gm = Total_protein_per_gm + temp_weight*protein_v[k]
668         Total_fat_per_gm = Total_fat_per_gm + temp_weight*fat_v[k]
669         Total_carbs_per_gram = Total_carbs_per_gram + temp_weight*carbohydrate_v[k]
670
671         Total_weight = Total_weight+temp_weight*100.
672
673         print(self.RecipeModel.recipes[k][0],', ', Total_calories_per_gm)
674         logger.debug('ingredient:%s' % self.panda_ingredients[k][2])
675         logger.debug('energy:%f' % energy_v[k])
676         logger.debug('protein:%f' % protein_v[k])
677         logger.debug('carbohydrate:%f' % carbohydrate_v[k])
678         logger.debug('fat:%f' % fat_v[k])
679     if(Total_weight == 0.):
680         return
681     temp_cal = Total_calories_per_gm/Total_weight
682     temp_fat = Total_fat_per_gm/Total_weight

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 13/21

```

683         temp_prot  = Total_protein_per_gm/Total_weight
684         temp_carb   = Total_carbs_per_gram/Total_weight
685         logger.debug('total calories per gram : %f' % temp_cal)
686         logger.debug('total carbs per gram : %f' % temp_carb)
687         logger.debug('total protein per gram : %f' % temp_prot)
688         logger.debug('total fat per gram : %f' % temp_fat)
689
690         # convert to string
691         string_cal = f"{temp_cal:.2f}"
692         string_fat = f"{temp_fat:.2f}"
693         string_prot = f"{temp_prot:.2f}"
694         string_carb = f"{temp_carb:.2f}"
695
696
697         #Update the values on the form
698         self.MRD.cal_line.setText(string_cal)
699         self.MRD.fat_line.setText(string_fat)
700         self.MRD.prot_line.setText(string_prot)
701         self.MRD.carb_line.setText(string_carb)
702
703         # put values onto form
704
705
706
707         self.new_recipe_record=(self.new_recipe_name,string_cal,string_prot,string_carb,string_fat,self.pack_record(self.panda_ingr
edients))
708         return
709
710     def add_row(self):
711         """ adds row to reipe table"""
712         row_position = self.RecipeModel.rowCount()
713         self.RecipeModel.insertRows(row_position)
714
715         return
716
717     def on_item_click(self, index):
718
719         #columns and rows start from 0
720         item_text = index.data()
721         item_row = index.row()
722         item_column = index.column()
723         return
724
725
726         #print(f"Clicked item: {item_text}")
727         #print(index.row())
728
729     def save_recipe(self):
730         """gets called from clicking the save button"""
731         record = self.RecipeModel.StoreValue()
732         logger.info("ingredients %s" % record)
733         #create original string
734         record = self.pack_record(record)
735         # Now update the record
736
737         #check if this is a new recipe or an updated one
738         if self.new_recipe_name != None:

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 14/21

```

739         rec_name=self.new_recipe_name
740         self.create_pandas_frame(name=rec_name)
741
742         self.insert_record()
743     else:
744         rec_name = self.selected_recipe
745         self.create_pandas_frame(name=rec_name)
746
747         self.update_record('recipes', 'ingredients', record, rec_name)
748
749         #self.new_recipe_name = None # reset recipe name
750         #here we call create_pandas_frame tframe for CalculateCalories start producing the
751
752         #remove window
753         #self.MRD.close()
754
755
756     return
757 def insert_record(self):
758     """inserts new recipe into table
759     columnname is a list of all the columns
760     record is a corresponding list"""
761     # create sql statement
762
763     record =self.new_recipe_record
764
765     # since ID is a non-zero number we need to find the highest number and then add one for the next reipce
766     sql = 'SELECT id from recipes;'
767     ID_number=[]
768     myquery = self.do_sql(sql)
769     while myquery.next():
770
771         ID_number.append(int(myquery.value(0)))
772     #find the highest number in the list
773     imax = max(ID_number)
774     new_recipe_id = str(imax+1)
775
776
777     b= new_recipe_id+',\'+record[0]+'\'','+record[1]+'','+record[2]+'','+record[3]+'','+record[4]+'','+record[5]+'\'\'
778
779
780     sql = 'INSERT INTO recipes (id,name,energy,protein,carbohydrate,fat,ingredients) VALUES (' +b+')';
781     logger.debug("insert recipe %s" % sql)
782     #finally exeute the sql
783     self.do_sql(sql)
784     return
785
786
787
788
789     #self.new_recipe_record=(self.new_recipe_name,string_cal,string_prot,string_carb,string_fat,self.pack_record(self.panda_ing
790     redients))
791
792     # the sql statements should be
793     # sql = INSERT INTO table1(name,energy,protein,carbohydrate,fat,ingredients) VALUES (row_name,record[0],record[1],record[2]
794     record[3],record[4]);
795     logger.debug(" insert statement %s % sql")

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 15/21

```

794         self.do_sql(sql)
795
796     return
797
798
799     def update_record(self, table, column_name, record, row_name):
800         """ updates record for name in table
801         example SQL statement
802         UPDATE recipes SET ingredients = '30 g almonds 60 g dry_tomatoes 517 g tenderloin' WHERE name='basil_tenderloin';
803         """
804
805         sql = 'UPDATE ' + table + ' SET ' + column_name + ' = \'' + record + '\' WHERE name = \'' + row_name + '\';'
806         self.do_sql(sql)
807
808     return
809
810     def pack_record(self, record):
811         """ put record back into original form for ingredients, which is just one string """
812         b = []
813         c = ''
814         #self.panda_ingredients are the correct list for creating the panda frame
815         self.panda_ingredients = a = record
816         for k in range(len(a)):
817             if (a[k][0] == ' '): break
818             b.append(a[k][0] + ' ' + a[k][1] + ' ' + a[k][2] + ' ')
819
820         for k in range(0, len(b)):
821             c = c + b[k]
822
823         # remove last space
824         d = c[0:len(c)-1]
825         return(d)
826
827
828     def getSelectedText(self, s):
829         self.selected_text = s
830         logger.info("You have selected %s text" % self.selected_text)
831         return
832
833
834     def getSelectedRecipe(self, s):
835         #Clear Buffer of recipes
836
837
838         self.RecipeModel.recipes = []
839
840         self.selected_recipe = s
841         logger.info("You have selected %s recipe" % self.selected_recipe)
842
843         # now we populate the listview
844         self.FillRecipeIngredientList()
845
846         # Now fill qlist
847         self.DisplayRecipeList()
848
849         # send signal list has been updated
850         self.RecipeModel.layoutChanged.emit()

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 16/21

```

851
852
853
854     return
855
856     def do_sql(self, sql):
857         """executes a sql statement"""
858         query = QSqlQuery(sql, db=self.mycal_db)
859         model = QSqlQueryModel()
860         logger.info("query in do_sql %s" % sql)
861         model.setQuery(query)
862         if query.lastError().isValid():
863             logger.error(f"Query error: {query.lastError().text()}")
864             # if this is a inserty error try replace.
865
866
867
868     return query
869
870
871     def FillRecipeIngredientList(self):
872         # here we get the ingredients from the Recipe database
873         # this is done with a query
874
875
876
877         sql = ' SELECT ingredients from recipes WHERE recipes.name LIKE \'' + self.selected_recipe + '\';'
878         query = QSqlQuery(sql, db=self.mycal_db)
879         model = QSqlQueryModel()
880
881         model.setQuery(query)
882         while query.next():
883
884             temp_value = ((query.value(0)))
885             # now we have to work with string
886             # the old database still had *u000a in front of the number
887             # need first to strip this
888             #check if temp contains *u000a
889             mask = 'u000a'
890             if(mask in temp_value):
891                 temp_new = self.StripUnicode(temp_value, mask)
892                 #temp_new = self.StripUnicode(temp_new, '*')
893
894
895
896             else:
897                 #already the new entry way
898                 temp_new1 = []
899                 temp_new = temp_value.split(' ')
900                 for k in range(0, len(temp_new), 3):
901                     temp_new1.append([temp_new[k], temp_new[k+1], temp_new[k+2]])
902                 self.recipe_ingredients = temp_new1
903                 return
904
905
906
907     delimiter = "g "

```


Mar 02, 2025 11:49

ControlDB_new.py

Page 17/21

```

908         result = temp_new.replace('u000a','')
909
910
911         result1 = self.split_and_keep(result, delimiter)
912         # strip last *
913         result11= result1[0]
914         result2 = result11[:len(result1[0])-1].split('*')
915         # we know that a string with underscores has a max of three spaces. Any number larger than this means words with spaces.
916         #in other words 4 spaces means the 3rd has to be replaced with _ 5 spaces mean the 3rd and 4th
917         result3 =[]
918         for a in result2:
919             space_indices = []
920             for i, char in enumerate(a):
921                 if char == ' ':
922                     space_indices.append(i)
923
924             if len(space_indices) == 4:
925                 na = a[:space_indices[2]] + '_' + a[space_indices[2] + 1:]
926
927                 result3.append(na)
928
929             elif len(space_indices) == 5:
930                 na = a[:space_indices[2]] + '_' + a[space_indices[2]+1:space_indices[3]] + '_' + a[space_indices[3] + 1:]
931                 result3.append(na)
932
933             else:
934                 result3.append(a)
935
936
937         recipe_string = result3
938         #finally split every field up into 3 '10 g bean' -> [10,'g','bean]
939         result4 =[]
940         for i in range(len(result3)):
941             temp = (result3[i].split(' '))
942             result4.append(temp[0:3])
943         self.recipe_ingredients = result4
944         return
945
946     def DisplayRecipeList(self):
947         """loop through ingredient list and add"""
948
949
950         for k in range(len(self.recipe_ingredients)):
951             print(self.recipe_ingredients[k])
952
953             self.RecipeModel.recipes.append(self.recipe_ingredients[k])
954         return
955
956
957     def split_and_keep(self,text, delimiter):
958         """deals with spaces instead of underscores"""
959         result = []
960         temp = ""
961         for char in text:
962             if char == delimiter:
963                 result.append(temp + char)
964                 temp = ""

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 18/21

```

965         else:
966             temp += char
967         if temp:
968             result.append(temp)
969         return result
970
971
972
973
974     def StripUnicode(self, mystring, mask):
975         newstring = mystring.replace(mask, '')
976         return newstring
977
978
979     def GetRecipeList(self):
980         """sql query to get all the names of the recipies
981         will be used to populate the recipe qlistbox"""
982         sql = 'SELECT name from recipes;'
983         query = QSqlQuery(sql, db=self.mycal_db)
984         model = QSqlQueryModel()
985
986         model.setQuery(query)
987         recipe_list = []
988         while query.next():
989             recipe_list.append(str(query.value(0)))
990
991
992         # now sort them alphabetically
993         items = [recipe_list[i] for i in range(len(recipe_list))]
994         items.sort(key=str.lower) #this way no diff between upper and lower case
995
996         return (items)
997
998
999
1000
1001
1002     def ModifyRecipe(self):
1003         pass
1004
1005
1006     def SizeWindow(self, window, myposit, mysize):
1007         """resizes the specified window ,where the parameters are two integer lists"""
1008         window.setGeometry(myposit[0], myposit[1], mysize[0], mysize[1])
1009         return
1010
1011     def calculate_portion_calory(self, myrecipe=None, myportion=0.):
1012         """ this calucaltes the calories of a portion for  given recipe,
1013         where the portion size is given in grams."""
1014
1015         # sql statement: SELECT enegry FROM recipes WHERE NAME = 'raspberry_tart';
1016         sql = 'SELECT energy FROM recipes WHERE NAME =\' ' + myrecipe + '\';'
1017         query = self.do_sql(sql)
1018         while query.next():
1019
1020             cal_per_gram = (query.value(0))
1021             portion_calory = float(cal_per_gram)*myportion

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 19/21

```

1022     logger.info("your total calories are %.1f " % portion_calory)
1023     return portion_calory
1024
1025
1026
1027
1028
1029 #####
1030 # section for signals and slots
1031
1032
1033 def Cancel(self,window):
1034     """ closes window without anything """
1035     window.close()
1036
1037
1038
1039
1040 def SaveIngredients(self):
1041     """saving the ingredients"""
1042     if( not self.swiss_foods):
1043         Ing_name      = self.name_label.text()
1044         Ing_calory     = float(self.calory_label.text())
1045         Ing_carb       = float(self.carb_label.text())
1046         Ing_fat        = float(self.fat_label.text())
1047         Ing_prot       = float(self.prot_label.text())
1048     else:
1049         Ing_name      = self.InTa_name
1050         Ing_calory     = self.InTa_energy
1051         Ing_carb       = self.InTa_carbohydrate
1052         Ing_fat        = self.InTa_fat
1053         Ing_prot       = self.InTa_protein
1054
1055
1056     # this is the record we got from the form input
1057
1058     #Check all fields are filled out
1059     if(Ing_calory !="" and Ing_carb !="" and Ing_fat!="" and Ing_prot != ""):
1060         self.Inform.destroy() # destrorys the window.
1061         #self.Inform.close() #
1062
1063     #now we need to add the values to the ingredients table
1064     # first load table
1065
1066
1067     self.ViewTable(self.CM.ingred_table,suppress_columns=self.ingredients_suppress_columns)
1068     self.model = QSqlQueryModel()
1069     self.table_view.setModel(self.model)
1070     #next we check we don't have an entry yet
1071
1072
1073     #sql = "SELECT name from ingredients WHERE ingredients.name LIKE 'Almond%' ; "
1074     sql = 'SELECT name from ingredients WHERE '+self.CM.ingred_table+'.name LIKE '+Ing_name+' ;'
1075     query = QSqlQuery(sql,db=self.mycal_db)
1076
1077     self.model.setQuery(query)
1078     while query.next():

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 20/21

```

1079         logger.error(' entry name already exists, try again %s' % query.value(0))
1080         self.CreateIngredientsForm1()
1081         #myvalue = query.value(0)
1082         # Now we need to add this ingredient to the table
1083
1084         self.record = [Ing_name, Ing_calory, Ing_carb, Ing_fat, Ing_prot]
1085         self.InsertRecord(table = self.CM.ingred_table)
1086         return
1087
1088
1089 if __name__ == '__main__':
1090     app = QApplication(sys.argv)
1091
1092     Title=None
1093     db_name='recipe_ak'
1094     db_user='klein'
1095     db_system='QPSQL'
1096     config_file = '/Users/klein/git/qt_exercises/config/config_mycal.json'
1097
1098     with open('/Users/klein/git/qt_exercises/config/pw.txt', 'r') as file:
1099         db_pwd = file.read().rstrip()
1100
1101
1102
1103
1104     suppress_columns = ['Sugar',
1105                         'Portions',
1106                         'Description',
1107                         'Saturated_fat',
1108                         'Fiber',
1109                         'Salt',
1110                         'Sodium',
1111                         'Ingredients',
1112                         'Time',
1113                         'Images',
1114                         'vegetarian']
1115     columnwidth = [40,200,200,200,200,200]
1116
1117     window = ControlDB_new(Title = "ControlDB",
1118                            db_name=db_name,
1119                            db_user=db_user,
1120                            db_system=db_system)
1121     #window.SetSize(800,500)
1122     #window.SetPosition(100,500)
1123
1124     window.setStyleSheet("background-color: white;")
1125     #window.CreateCalendar()
1126
1127
1128
1129     #window.show()
1130     window.ConnectDataBase()
1131     window.ShowTables()
1132     #window.calculate_portion_calory(myrecipe = 'raspberry_tart',myportion=200.)
1133     #window.ViewTable('recipes', suppress_columns=suppress_columns, columnwidth=columnwidth)
1134
1135     window.MyRecipes_new()

```

Mar 02, 2025 11:49

ControlDB_new.py

Page 21/21

```
1136 #window.CreateIngredientsForm1()  
1137  
1138 # now run the app  
1139     app.exec()
```